

Comprehensive Briefing: Relational Algebra and Join Operations in DBMS

Summary

This briefing presents a **structured and exam-oriented synthesis** of **Relational Algebra (RA)** and **Join Operations** in Database Management Systems. Relational Algebra forms the **mathematical and theoretical foundation of SQL**, defining *how* data is retrieved, while SQL focuses on *what* data is required.

Key highlights include:

- **Classification of query languages** into procedural and non-procedural paradigms
- **Basic vs. Derived Relational Algebra operators** and their mathematical constraints
- **Set operation requirements** (arity and domain compatibility)
- **Operator properties** such as commutativity and reorderability
- **Join operations**, their internal mechanics, and differences
- **Outer joins** as a solution to the dangling tuple (data loss) problem

1. Classification of Query Languages

Relational query languages differ in how explicitly they define execution steps.

Query Language Types

Type	Representative Language	Focus
Procedural	Relational Algebra	Specifies what to retrieve and how
Non-Procedural	SQL, TRC, DRC	Specifies what to retrieve only

Key Insight:

Understanding Relational Algebra is essential for mastering SQL, as SQL query optimizers internally translate queries into RA-like execution plans.

2. Basic Relational Algebra Operators

Basic operators are the **primitive building blocks**. All other operators can be expressed using them.

2.1 Selection (σ) and Projection (π)

Selection (σ)

- **Row-oriented (horizontal filtering)**
- Returns complete tuples that satisfy a condition

Example:

$\sigma_{\{\text{Marks} \geq 9\}}(\text{Student})$

Projection (π)

- **Column-oriented (vertical filtering)**
- Returns only specified attributes
- Automatically removes duplicate rows

Example:

$\pi_{\{\text{Name}, \text{CGPA}\}}(\text{Student})$

2.2 Cross Product (Cartesian Product $\times$)

The Cross Product combines **every tuple of one relation with every tuple of another**.

Mathematical Properties

Property	Rule
Columns (Degree)	$\text{Degree}(R \times S) = \text{Degree}(R) + \text{Degree}(S)$
Rows (Cardinality)	$\text{Card}(R \times S) = \text{Card}(R) \times \text{Card}(S)$

Common Attributes

- If both tables have attribute **C**, both are retained

- Distinguished using prefixes: **R.C**, **S.C**

△ Cross Product alone is rarely useful; it is mainly used **internally for Join operations**.

2.3 Set Operators

Set operators require **strict compatibility**.

Compatibility Rules

1. **Arity Compatibility** – Same number of attributes
2. **Domain Compatibility** – Corresponding attributes must have the same data type

Set Operators Summary

Operator	Symbol	Commutative?
Union	U	Yes
Intersection	∩	Yes
Set Difference	–	No

Example:

$R - S \neq S - R$

3. Commutativity and Operator Properties

Understanding operator behavior is critical for query optimization and transformations.

Operator Behavior

Operator	Property
Selection	Fully commutative
Projection	Not strictly commutative
Cross Product	Commutative (column order may differ)

Selection Reordering

$$\sigma_{\{C1\}}(\sigma_{\{C2\}}(R)) = \sigma_{\{C2\}}(\sigma_{\{C1\}}(R))$$

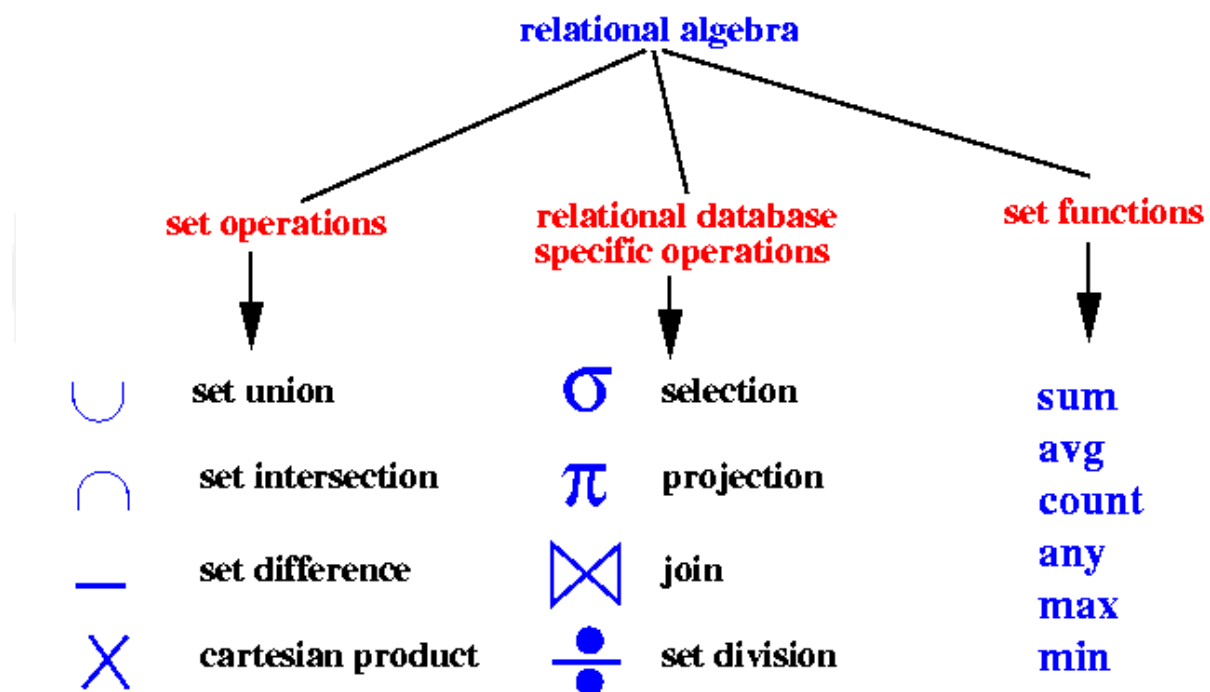
Projection Caution

Projection may discard attributes required by later operations, so reordering is **not always valid**.

4. Join Operations (Derived Operators)

Join operations are **derived** because they combine:

Cross Product + Selection



- Relations r, s :

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

■ $r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ

R

A	B
1	2
4	5

S

B	C
2	3
6	7

R NATURAL FULL JOIN S

A	B	C
1	2	3
4	5	NULL
NULL	6	7

Expression	Meaning
R, S	$R \times S$
R cross join S	
R natural join S	$R \bowtie S$
R join S on Condition	$R \bowtie_{condition} S$

4.1 Inner Joins

Inner joins return **only matching tuples**.

Types of Inner Joins

Join Type	Condition	Duplicate Attributes
Natural Join	Equality on common attributes	Removed
Equi-Join	Equality (=)	Retained
Conditional Join	Any comparison (<, >, ≠, =)	Retained

Natural Join

- Automatically matches attributes with the same name
- Eliminates duplicate join columns

Equi-Join

- Uses equality condition
- Keeps both joining attributes

Conditional Join

- Most flexible
- Allows non-equality comparisons

5. Dangling Tuples Problem

A **dangling tuple** is a row that:

- Has no matching tuple in the other table
- Gets **excluded** during an inner join
- Causes **loss of information**

This is a major limitation of inner joins.

6. Outer Joins

Outer joins solve the dangling tuple problem by **preserving unmatched rows** and filling missing values with **NULL**.

Types of Outer Joins

Join Type	Preserves
Left Outer Join (LOJ)	All tuples from left table
Right Outer Join (ROJ)	All tuples from right table
Full Outer Join (FOJ)	All tuples from both tables

Behavior Summary

- Inner Join → Only matches
- LOJ → Matches + left dangling tuples
- ROJ → Matches + right dangling tuples
- FOJ → Matches + all dangling tuples

7. Rename Operator (ρ)

The Rename operator improves **readability, disambiguation, and query composition**.

Rename Capabilities

- Rename a relation
- Rename all attributes
- Rename specific attributes
- Rename relation + attributes together

Example:

$\rho_{\text{Emp}}\{\text{Emp}(\text{Eid}, \text{Name}, \text{Salary})\}(\text{Employee})$

